

First steps into Git and Gitlab

- An overview of Gitlab
- Basic actions with git
- Secure data and easy collaboration

An overview of GitLab

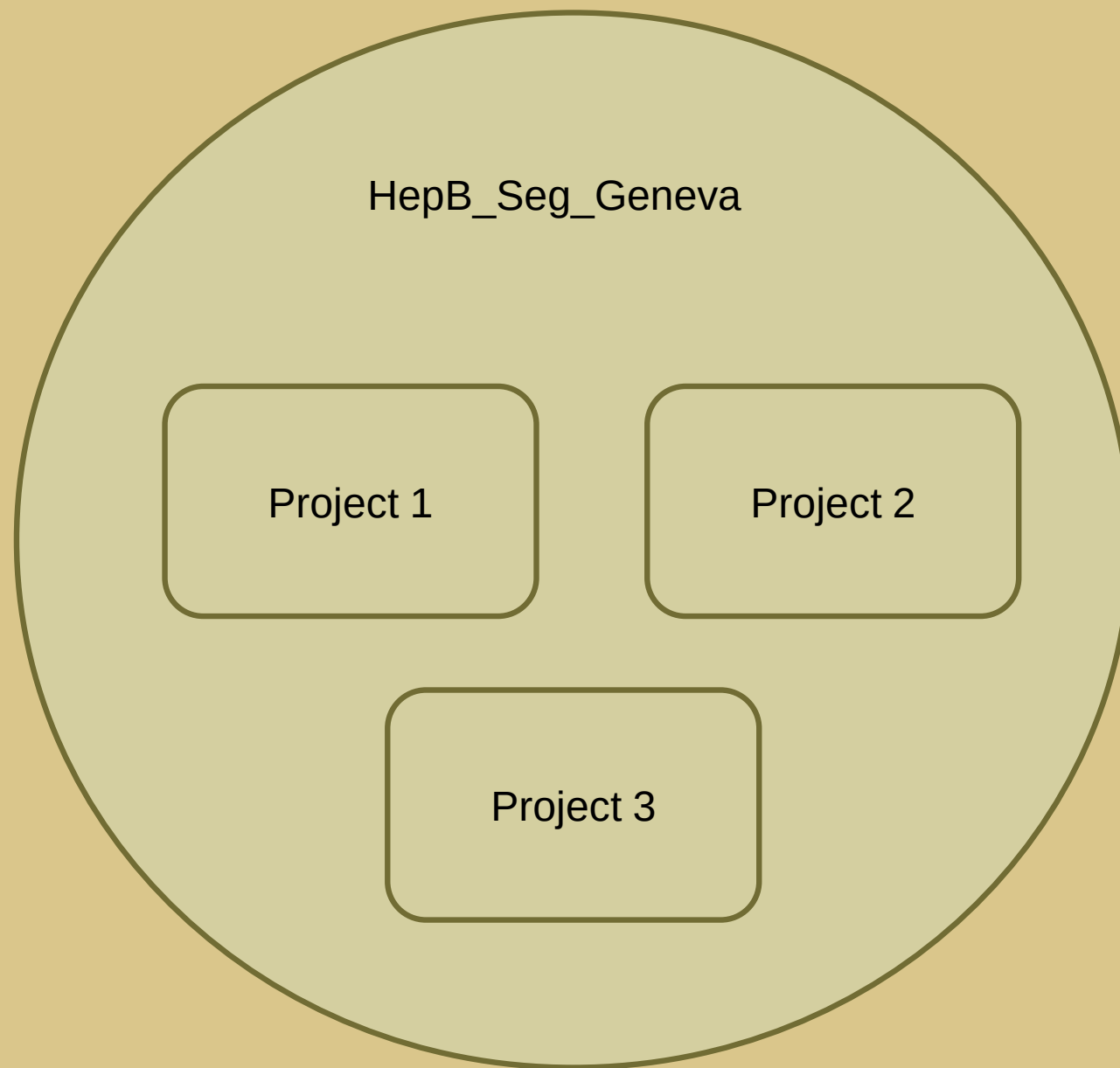


A Project can contain a lot of different scripts and be own by multiple persons.

It's NOT a project by person with all your work inside. For each of your analysis not related create a new project.



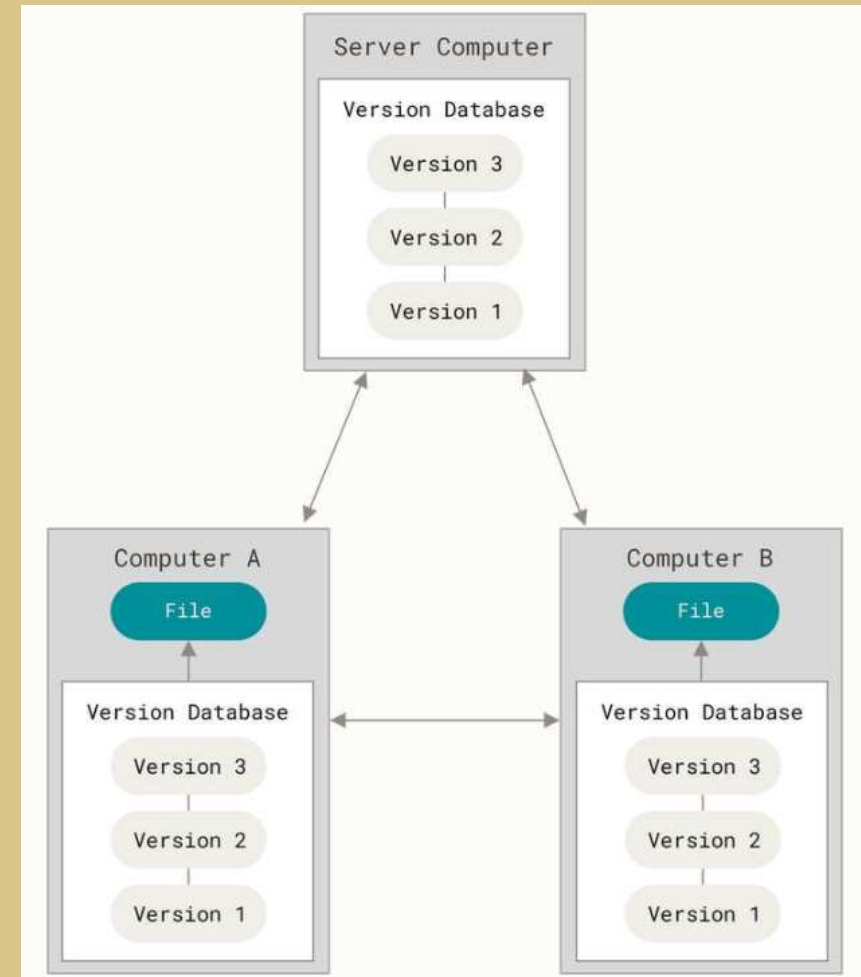
Our Group is private so every Project inside it



A **Repo** (repository) is your project folder. It can be either local or remote.

A **Local Repo** is the version of the folder on your local computer.

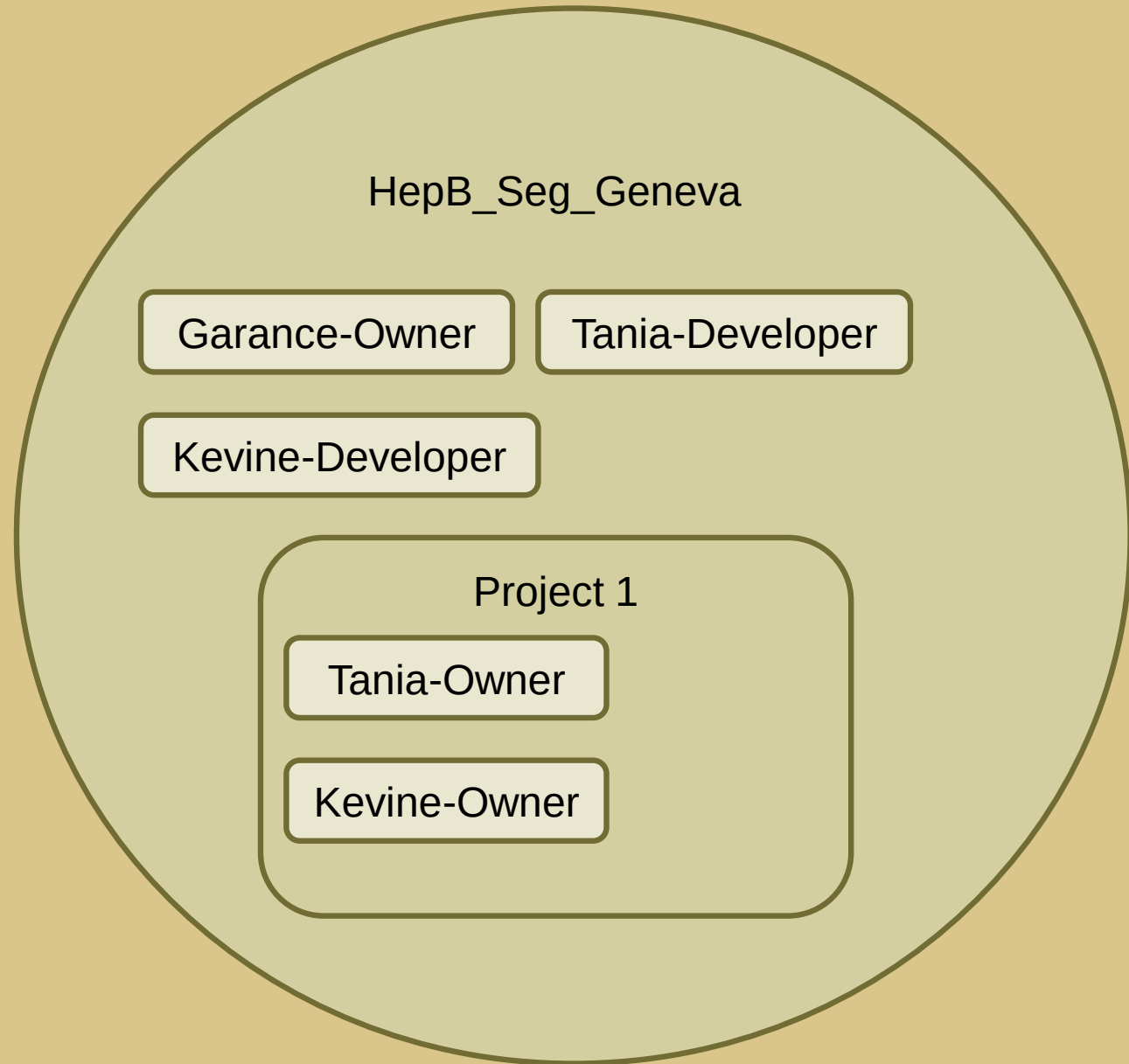
A **Remote Repo** is the version of the folder on online service like GitLab UniGe. Sometimes called **origin**.



Roles

- Owner
- Maintainer
- Developer
- ...

When you create a new Project, invite people who will work with you as owner of the project



Demo

Basic actions with Git

Action 1: Create a new repository

Action 2: Clone an existing repository

Action 3: Pull an updated repository

Action 4: Make a change, commit and push

Action 5: Go back to an old commit (bonus)

This actions can be found on the Git Survival Guide, with more information and easier to copy-paste !

Action 1: Create a new repository on GitLab

- Go to the group and click on **create project**.
- Click on **create blank project**.
- Write the name of the project and **uncheck the README**.
- Click on **create project**.
- If you will work with other people, click on **members** then **invite members**.
- Add the email address and give the **owner** role.

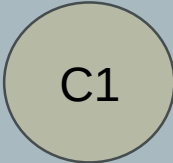
Action 2: Publish an existing repository

- Do Action 1
- Go to the root of your repository in a terminal

```
git init
git add .
git commit -m "First Publication"
git remote add origin ssh-path-of-the-project
git branch -M main
git push -u origin main
```



A commit is like
a snapshot of
your repository



C1

Action 3: Clone an existing remote repository

- Open a terminal and go where you want to add the new repository

```
git clone PASTE_THE_SSH_URL_HERE
```

Action 4: Pull latest update

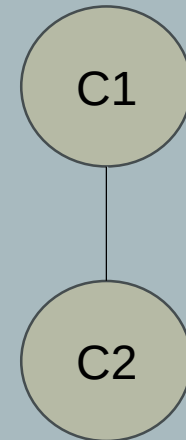
- Open a terminal and go to the root of your repository

```
git pull origin main
```

Action 5: Make a change, commit and push

- Do a git pull !!! Just in case
- Make your change

```
git add .  
git commit -m "message explaining the change"  
git push origin main
```



Action 6: Go back to another commit

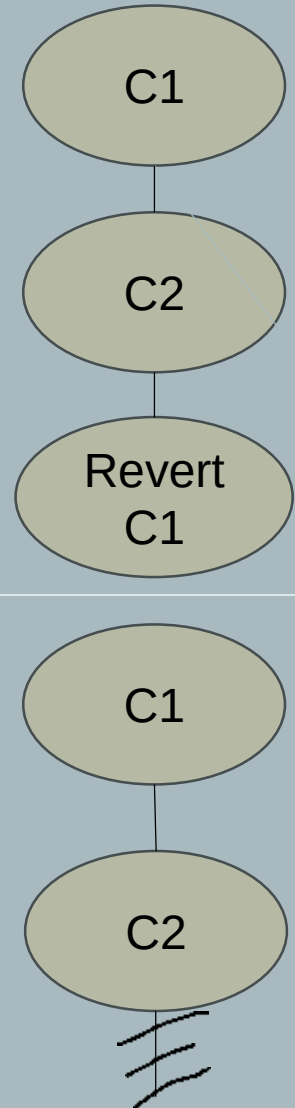
- If you don't have done any change since your last commit:

```
git revert <hash-last-commit>
```

```
git revert <ancient-hash>..<recent-hash>
```

- If you don't have done any change since your last commit:

```
git restore --staged --worktree .
```



Secure data and easy collaboration

Your `read_csv()` function take a path and it's relative to the organizations of your files.

So, you have to change it each time the code is given to a new developer.

```
library(tidyverse)

data <- read_csv("data/my_file.csv")

head(data)
str(data)
summary(data)

data_clean <- data %>%
  drop_na() %>%
  filter(age > 0)

data_summary <- data_clean %>%
  group_by(group) %>%
  summarise(
    mean_age = mean(age),
    n = n()
  )

print(data_summary)
```

Now we never have to change the script because it will take the path in the .env file

```
library(tidyverse)
library(dotenv)

load_dot_env()
data <- read_csv(Sys.getenv("DATA_PATH"))

head(data)
str(data)
summary(data)

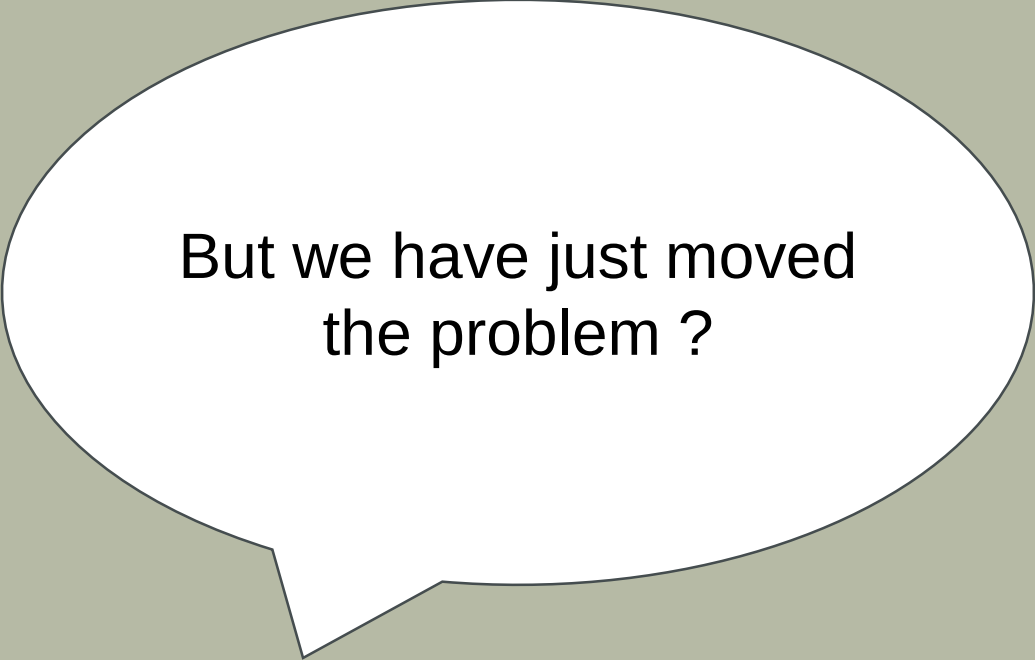
data_clean <- data %>%
  drop_na() %>%
  filter(age > 0)

data_summary <- data_clean %>%
  group_by(group) %>%
  summarise(
    mean_age = mean(age),
    n = n()
  )

print(data_summary)
```

.env · ENV

1	DATA_PATH=data/my_file.csv
2	



But we have just moved
the problem ?

No because the .env will not be share between people !

That's why we will add a .gitignore to tell git to keep all the kind of files inside locals !

And because you are a nice person, you will also create a .env.example to explain to other people which kind of .env are expected. This one will be shared.

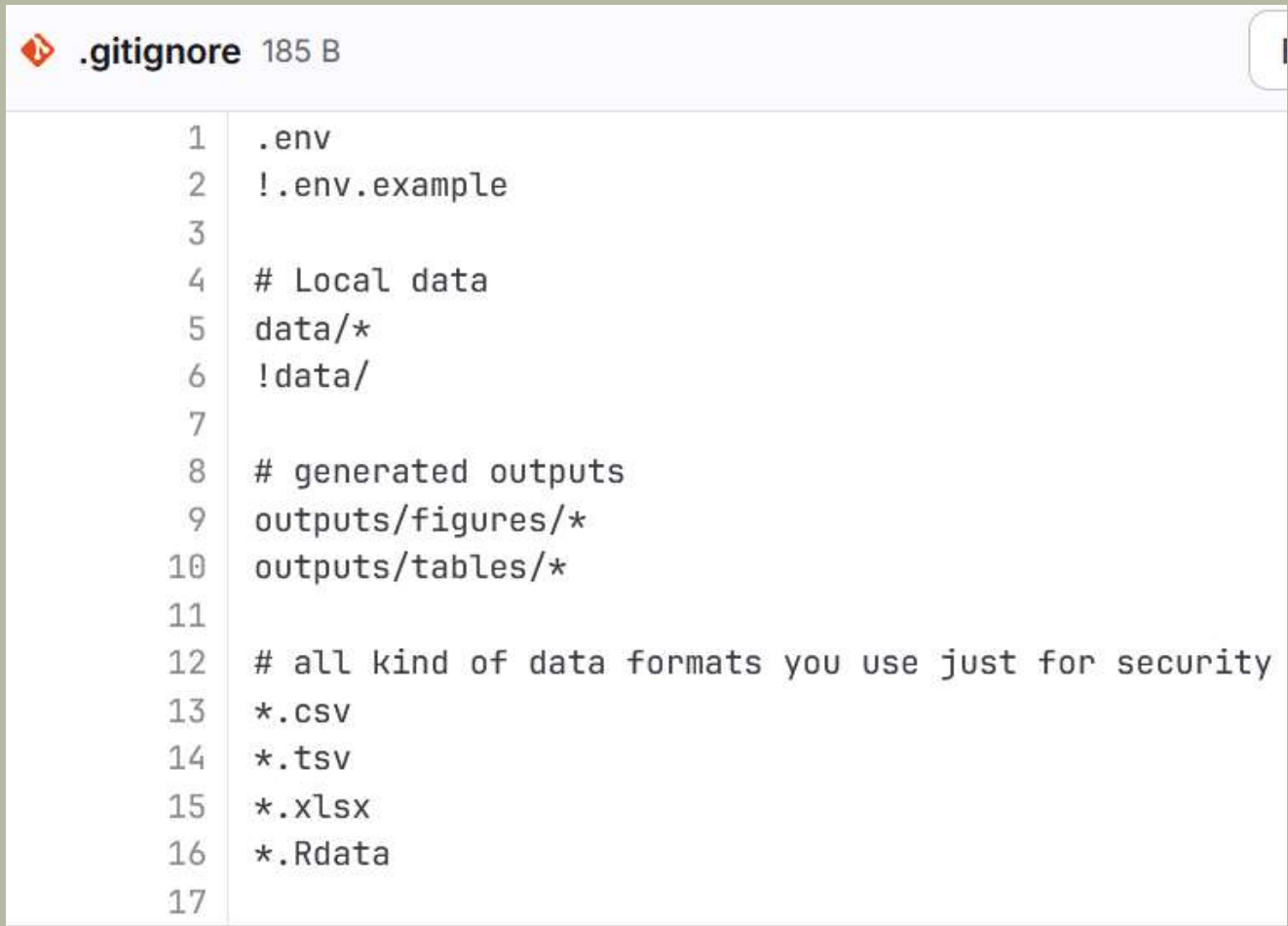
.gitignore · GITIGNORE

```
1  .env
2  data/*
3  *.csv
4  *.Rhistory
5  *.RData
6  *.Rproj.user/
7
```

.env.example 66 B

```
1  DATA_PATH=path/to/your/file.csv
2
3  MY_API_KEY=enter_an_api_key
4
5
```

Into the .gitignore we will also put all the files that should not be push remotely as data, figures, tables or sensitive information (passwords etc)

A screenshot of a code editor showing a .gitignore file. The file name ".gitignore" is in the top bar with a size of 185 B. The file content is as follows:

```
1 .env
2 !.env.example
3
4 # Local data
5 data/*
6 !data/
7
8 # generated outputs
9 outputs/figures/*
10 outputs/tables/*
11
12 # all kind of data formats you use just for security
13 *.csv
14 *.tsv
15 *.xlsx
16 *.Rdata
17
```

My_sub_project/

- .gitignore
- .env # never committed (contains secrets) or .Renv for R
- .env.example # committed, as a model or .Renv.example for R
- renv.lock # packages versioning (reproducibility)
- R/
 - 01_import.R
 - 02_clean.R
 - 03_modele.R
- data/
 - raw/ # never committed
 - processed/ # never committed
- outputs/
 - figures/ # never committed (except last push)
 - reports/
 - analyse.Rmd



IN CASE OF FIRE



git commit



git push



exit building

Next steps

- Publish your current work on repository into the group. This is an important step !
- Follow the incoming last guide on branches to master git and merge conflict.
- Last session will be a recap on branch and good practices.